

Transforming SysML v1 (DELS) into Many Small SysML v2 Artifacts

XML $\rightarrow$ IR $\rightarrow$ Deterministic v2 $\rightarrow$ 200–500 line Shards

Creatix Chu (Georgia Tech)
creatixchu@gatech.edu

October 8, 2025

<https://github.com/usnistgov/DiscreteEventLogisticsSystems>

Objective

- Convert large SysML v1 models (.mdzip, .xml) to high-quality SysML v2 *textual* artifacts.
- Produce **many small, semantically coherent files** (target: 200–500 lines each).
- Generate paired **NL summaries** for each file for a SysMLv2–NL dataset.
- Keep the process **deterministic** and **auditable**; use LLMs only for stylistic polish.

Pipeline Overview

- ➊ **Ingest v1**: unzip .mdzip/parse XML; normalize profiles/stereotypes.
- ➋ **Extract IR**: JSON intermediate representation (structure, behavior, requirements).
- ➌ **Rule-based mapping**: deterministic v1→v2 text.
- ➍ **Sharding**: split into 200–500 line cohesive files.
- ➎ **LLM pass**: refine style, not semantics.
- ➏ **Validation**: syntax + semantic checks, imports/exports.
- ➐ **Packaging**: NL summaries + manifest (.jsonl).

Intermediate Representation (IR)

```
{  
  "blocks": [{  
    "id": "b:WarehouseSystem",  
    "name": "WarehouseSystem",  
    "parts": [{"name": "conveyor", "type": "Conveyor"}],  
    "valueProperties": [{"name": "throughput", "type": "item/s"}],  
    "ports": [{"name": "cmdIn", "kind": "proxy", "interface": "CmdIF"}]  
  }],  
  "requirements": [{"id": "1.2", "name": "Performance", "text": "..."}],  
  "activities": [{"name": "InboundShipment", "actions": [...]}]  
}
```

v1→v2 Mapping Rules

Structure

- Block → part def
- Part property → part membership
- Value property → attribute/constraint
- Connector → connection

Requirements

- Requirement → requirement def
- Links → v2 allocations/satisfy relations

Behavior

- Activity → action defs
- Sequence → occurrences/messages
- Use case → usecase defs

Stereotypes

- → v2 metadata / #keywords

Sharding Strategy (200–500 lines each)

- Split by concern: Structure / Behavior / Requirements / Interfaces.
- Within each concern, anchor on top block or scenario root.
- Greedy BFS add elements until ~ 400 lines.
- Record exports/imports.
- Keep large units/enums in shared Libraries/.

Example Generated v2 Chunk

```
package Structure::WarehouseSystem;
import Libraries::Units;

part def WarehouseSystem;
WarehouseSystem {
  part conveyor: Conveyor;
  attribute throughput: Real; // unit: item/s
  port cmdIn: In<CmdIF>;
  connection cmdIn -> ControlStation.cmdOut using item Cmd;
}
```

Output Dataset Organization

- Each file = one coherent SysMLv2 chunk (200–500 lines)
- Paired natural-language summary
- **Manifest (.jsonl)** entry:

```
{  
  "id": "Structure/Warehouse.CoreParts",  
  "sysml2_path": "SysMLv2/Structure/Warehouse.CoreParts.sysml",  
  "nl_path": "NL/Structure/Warehouse.CoreParts.md",  
  "lines": 372,  
  "exports": ["WarehouseSystem"],  
  "imports": ["AGV", "Pallet"]  
}
```


LLM Role (Polish Only)

- Input: deterministic v2 + IR snippet
- Output: stylistically refined v2 text or short NL summary
- Guardrail prompt:
“Do not invent new elements. Only reorganize or reformat existing definitions for v2 idiom.”

Validation & Expert Feedback Points

- Check mapping fidelity vs. v1 semantics.
- Confirm v2 package layout and naming best practices.
- Discuss trade-offs: *satisfy* vs. *allocation usage*.
- Recommend validation checks for consistency across shards.

Summary

- Deterministic pipeline: v1 XML $\rightarrow$ IR $\rightarrow$ v2 $\rightarrow$ 200–500 line shards.
- NL summaries and manifests for dataset creation.
- Ready for expert feedback on mapping conventions.